Retroalimentación para P1 — secciones 1 a 3

Campo	Valor
Revisado	`src/migue/p1.ipynb`
Alcance	Secciones 1 (configuración), 2 (datos y EDA) y 3 (features y split)
Fecha	9 de noviembre de 2026
Veredicto	**Aprobado con cambios menores**

Veredicto

El notebook está bien construido: las 30 celdas (14 de código) se ejecutan en orden, sin errores, y no hay fuga de información. La auditoría de 3.6 (nueve controles) es la parte más valiosa y demuestra con cálculos lo que otros notebooks solo afirman.

El problema no es la validez técnica, es el **exceso**. Hay **410 líneas de código** para preparar datos, más de lo que la rúbrica pide y más de lo que el equipo necesita mantener. La meta es bajar a **~250 líneas** sin perder ningún control importante. La práctica se califica por 3.1 (optimización), 3.2 (predicción) y 3.4 (conclusiones), no por la profundidad del EDA.

---

1. Aclaraciones de datos

1.1 Los nulos son estructurales, no datos faltantes

El dataset original **no tiene nulos**. Los 90 nulos aparecen en `ventas_features.csv`, que es un archivo derivado de la práctica 001:

Archivo	Filas	Nulos	Qué es
`sales_history.csv`	600	**0**	Ventas crudas (producto, fecha, cantidad, stock)
`inventory.csv`	10	**0**	Catálogo de productos
`ventas_features.csv`	600	**90 celdas en 70 filas**	Derivado con rezagos y promedio móvil

Columna	Nulos	Origen
`ventas_dia_anterior`	10	Día 1 de cada producto: no existe el día anterior
`promedio_movil_7d`	10	Día 1: no hay días previos que promediar
`ventas_hace_7_dias`	70	Días 1 a 7 de cada producto: no existe "hace 7 días"

Son el **periodo de calentamiento de los rezagos**. El día 1 tiene 3 nulos, los días 2–7 tienen 1, y por eso son 90 celdas repartidas en 70 filas. Como `sales_history.csv` empieza el 1-ene-2022, no hay historia anterior para rellenarlos sin inventar ventas. Por eso `dropna` es correcto: quedan 530 filas (53 días por producto).

Qué cambiar en tu notebook:

1. Reescribir la idea: **"nulos estructurales de los rezagos"**, no "datos faltantes".
2. Añadir estos tres `assert` en 2.2:

```
crudo = pd.read_csv(RUTA_DATOS.with_name("sales_history.csv"))
assert crudo.isna().sum().sum() == 0                    # el dato original no tiene nulos
assert df[COLS_LAG].isna().sum().sum() == 90            # 90 celdas de calentamiento
assert df[COLS_LAG].isna().any(axis=1).sum() == 70      # concentradas en 70 filas
```

1. No cambiar el `dropna` ni imputar. Imputar inventaría ventas que no ocurrieron.

1.2 Categorías: cinco de seis tienen un solo producto

No es que todas las categorías tengan un producto: **5 de 6 tienen uno solo** y **Alimentos tiene 5**. Los datos lo confirman y la estructura no cambia al quitar nulos (cada producto queda con 53 filas):

Categoría	Productos	IDs
Alimentos	5	P0001–P0005
Bebidas	1	P0008
Cuidado personal	1	P0007
Hogar	1	P0010
Limpieza	1	P0006
Snacks	1	P0009

Consecuencia: `categoria = f(product_id)`. Sus 5 dummies son idénticas a las de `product_id`; la de Alimentos es la suma de P0001–P0005. `categoria` **no aporta información nueva al modelo.** No es un error, pero cuesta 5 columnas y duplica el peso de la identidad del producto en la distancia del kernel RBF.

Decisión de equipo (no unilateral):

- **Opción A (recomendada, más simple):** `CAT = ["product_id"]`. `X` pasa de 25 a 20 columnas, sin perder información.
- **Opción B:** mantener las dos y documentar la duplicación. No rompe nada, pero hay que decir por qué se conserva.

Si se elige A, actualizar `assert X.shape == (530, 20)` y la tabla de variables.

---

2. Acciones priorizadas

#	Prio	Dónde	Acción	Criterio de aceptación
1	**P0**	3.2	Quitar `df_crudo = df`; cargar `df_crudo` una sola vez	Re-ejecutar 3.2 y luego 3.6 no rompe
2	**P0**	2.2	Reescribir la explicación de nulos + los 3 `assert`	Queda claro que son estructurales

#	Prio	Dónde	Acción	Criterio de aceptación
3	**P0**	Equipo	Decidir `CAT = ["product_id"]` (20 columnas) o mantener 25	Decisión escrita en el plan y asserts actualizados
4	**P0**	Celda 0	Corregir "secciones 4 a 12" → **"4 a 11"**	Coincide con el plan
5	P1	Celda 15	Bajar de 59 a ≤25 líneas: `MOTIVOS` largo a tabla Markdown	Código compacto, justificaciones conservadas
6	P1	Celda 4	Sustituir `extremo()` por `df.agg(["min","max"]).T`	35 → ≤18 líneas
7	P1	Celda 6	Simplificar el heatmap (`cmap="Oranges"` , sin colormap propio)	41 → ≤25 líneas
8	P1	Figuras	Máximo 4: nulos, distribución, categoría, temporal	`split_temporal.png` se elimina o se justifica
9	P1	Celda 28	Mover el ensayo al informe; dejar 5 bullets	≤8 líneas
10	P1	2.3	Etiquetar "600 filas (EDA)" vs "530 (modelado)"	Sin cifras contradictorias
11	P2	`docs/plan`	Añadir `semana_iso` , `eda_nulos.png` , `tabla_variables.csv`	Plan coherente con el notebook
12	P2	Entorno	Fijar Python 3.13 (o 3.12) y congelar `requirements.txt`	Versiones reproducibles
13	P2	Git	`git rm --cached .~lock.guia.docx#` + `.gitignore`	Archivo temporal fuera del repo

Tiempo estimado: **1,5–2 h**.

---

3. Correcciones con código

3.1 `df_crudo` re-ejecutable

```
df_crudo = pd.read_csv(RUTA_DATOS, parse_dates=["fecha_venta"])
df_crudo = df_crudo.sort_values(["fecha_venta", "product_id"]).reset_index(drop=True)
df = df_crudo.copy()                        # EDA sobre 600

# ... más adelante, sin volver a tocar df_crudo:
df = df_crudo.dropna(subset=NUM + [TARGET]).reset_index(drop=True)   # 530
```

Hoy `df_crudo = df` dentro de 3.2 hace que, si alguien vuelve a ejecutar solo esa celda, `df_crudo` quede con 530 filas y la auditoría de 3.6 falle con un `assert` confuso.

3.2 Tabla de variables compacta (celda 15)

```
NUM = ["mes", "dia_del_mes", "dia_semana", "es_fin_de_semana", "es_feriado",
       "semana_del_periodo", "ventas_dia_anterior", "ventas_hace_7_dias",
       "promedio_movil_7d", "stock_en_momento", "dias_para_expirar"]
CAT = ["product_id"]          # o ["categoria", "product_id"] si el equipo lo decide
TARGET = "cantidad_vendida"

EXCLUIDAS = {
    "stock_actual": "constante por producto",
    "nombre": "1 a 1 con product_id",
    "unidad": "fijo por producto",
    "fecha_venta": "solo ordena y divide",
    "anio": "constante",
    "nombre_dia": "1 a 1 con dia_semana",
    "tipo_dia": "derivada de es_fin_de_semana y es_feriado",
    "semana_iso": "no es monótona (1-ene = semana 52)",
}
assert set(NUM) | set(CAT) | set(EXCLUIDAS) | {TARGET} == set(df.columns)
```

Las justificaciones largas van en una tabla Markdown encima; el código solo verifica cobertura. Eso cumple el criterio "cada exclusión justificada por escrito" sin 40 líneas de diccionario.

3.3 Validación compacta (celda 4)

```
resumen = df.agg(["min", "max"]).T
resumen.insert(0, "tipo", df.dtypes.astype(str))
resumen.insert(1, "nulos", df.isna().sum())
resumen.insert(2, "distintos", df.nunique())
resumen
```

Probado con el CSV real: funciona con columnas de texto, fecha y numéricas. Elimina el helper `extremo()` y doce líneas.

3.4 Mapa de nulos simplificado (celda 6)

```
mapa = (
    df.assign(n=df[COLS_LAG].isna().sum(axis=1))
    .pivot(index="product_id", columns="fecha_venta", values="n")
)
fig, ax = plt.subplots(figsize=(11, 4))
sns.heatmap(mapa, cmap="Oranges", linewidths=0.3, linecolor="white",
            cbar_kws={"label": "rezagos sin historial"}, ax=ax)
ax.set(title="Nulos estructurales al inicio de cada producto", xlabel="Fecha", yla-
bel="Producto")
guardar(fig, "eda_nulos.png")
plt.show()
```

El colormap propio y las etiquetas personalizadas no aportan; el mensaje es el mismo.

3.5 Markdown de 3.7 (celda 28), de 21 líneas a 5

```
- `es_feriado` vale 0 en train: los dos feriados caen en test y el modelo no aprende su efec-
to.
- `mes`, `semana_del_periodo` y `dias_para_expirar` salen del rango de train: riesgo de extra-
polación.
- 5 dummies de `categoria` duplican las de `product_id`: decisión de equipo (ver 3.1).
- Test = 100 filas / 10 días: métricas ruidosas, reportar también RMSE de validación cruzada.
- Los rezagos usan ventas reales del día anterior: es pronóstico a 1 día, no a varios.
```

---

4. Dónde está el exceso

Celda	Tema	Líneas	Objetivo
2	Configuración	40	≤25
4	Validación	35	≤18
6	Mapa de nulos	41	≤25
10	Categoría y producto	32	≤30
12	Evolución temporal	27	≤27
15	Tabla de variables	**59**	≤25
21	Split	25	≤25
25	Auditoría de fuga	44	≤35
27	Rangos y duplicadas	20	≤20
28	Markdown de hallazgos	21	≤8
Total código		**410**	**~250**

Las tres celdas que más pesan son 15, 25 y 6. En 15 el problema es el diccionario de motivos; en 25, la redacción; en 6, la personalización del gráfico.

---

5. Reglas de estilo para el resto del notebook

1. Una celda = una idea; **≤ 40 líneas** de código.
2. Si un helper se usa una sola vez, no es helper: escríbelo inline.
3. Prefiere `agg`, `groupby`, `pivot_table` y `describe` antes que bucles manuales.
4. `assert` solo para invariantes que podrían romperse con datos nuevos. No para repetir literales (`len(NUM) == 11` no protege de nada).
5. Markdown **≤ 8 líneas** por celda; el ensayo va al informe, no al notebook.
6. **4 figuras máximo** en el EDA; cada una debe responder una pregunta.
7. Nombres en español, `snake_case`, sin abreviaturas.
8. Cero warnings, cero celdas muertas, ejecución total < 2 min.
9. No repitas tablas del plan: referencia la sección.
10. Los números que aparezcan en Markdown deben salir de una celda, no estar escritos a mano.

---

6. Lo que NO se toca

- **La auditoría de fuga de 3.6** (los nueve controles). Es la parte más valiosa del notebook; solo comprime la redacción.
- El split temporal, `COLUMNAS` y la exclusión de `semana_iso` (bien detectada; falta en el plan).
- `guardar()`: se usa cinco veces, se queda.
- Los `assert` de forma, duplicados, nulos y proporción del split.

---

7. Checklist de entrega

- □ Re-ejecutar 3.2 y después 3.6 no falla.
- □ Verificado que `sales_history.csv` tiene 0 nulos y explicados los 90/70.
- □ Decisión de `categoria` registrada; `X` con 20 o 25 columnas y asserts coherentes.
- □ Celda 4 ≤ 18 líneas; celda 15 ≤ 25; celda 6 ≤ 25; celda 28 ≤ 8.
- □ 4 figuras EDA máximo.
- □ "Secciones 4 a 11".
- □ Plan actualizado con `semana_iso`, figuras y `tabla_variables.csv`.
- □ 0 errores, 0 warnings, ejecución < 2 min.

---

8. Lo que ya está bien (no rehacer)

- Las verificaciones son reales, no afirmaciones: dimensiones, rangos, nulos, constancia de columnas y proporción del split.
- La auditoría de 3.6 reconstruye los rezagos, demuestra que `promedio_movil_7d` no incluye el día actual y que `stock_en_momento` no descuenta las ventas del propio día.
- El split es limpio: train 430 filas / 43 días (8-ene → 19-feb), test 100 filas / 10 días (20-feb → 1-mar), sin solapamiento y con los 10 productos en ambos.
- `semana_iso` está bien excluida: el 1-ene-2022 es semana ISO 52 y rompe el orden.
- El diagnóstico de extrapolación (`es_feriado` constante en train, 40 % de `semana_del_periodo` fuera de rango) es información útil para P2 y para las limitaciones de P4.
- El contrato `X_train` , `X_test` , `y_train` , `y_test` , `COLUMNAS` , `SEED` y `guardar()` está claro y verificado.